Report on AHB lite to APB4 Bridge Implementation along with Multi-slave Interconnect including Address Decode Logic and Error Propagation

Submitted By

Mirza Ateef Baig

EMP ID 6911

Introduction

Modern computer systems rely more and more on On-Chip communication protocols for the fast data transfer or to establish communication between different modules and to exchange data. In General, we have two types of protocols present

1. On-Chip Protocols
2. Peripheral Protocols or Off Chip Protocols

On-Chip Protocols: These protocols in VLSI are standardized rules that manage internal data transfer between different intellectual property (IP) blocks, processors, and memory modules inside a System-on-Chip (SoC). Common examples include ARM AMBA (AXI, AHB, APB), Open Core Protocol (OCP), Wishbone, and TileLink. This protocol is used to communicate between the components of an SoC.

Peripheral Protocols: These protocols in VLSI are used for Slower Modules or designs where the working frequency is less and main idea is reduced power consumption these protocols are used for communication between external devices like UART, I2C, SPI, TIMER etc. this protocol is used when the components of an SoC is going to communicate with the peripheral devices.

The AMBA AHB (Advanced High-Performance Bus) is a part of ARM AMBA (Advanced Microcontroller Bus Architecture) family designed to support High performance, high Bandwidth communication between different components in a System on Chip (SoC).

The AHB provides a single clock edge Operation, Burst Transfer and a pipelined structure making it suitable for High-Speed Designs.

AMBA (Advanced Microcontroller Bus Architecture) is a set of specification developed by the ARM to standardize the communication between different functional blocks of an SoC.

Few Protocols are:-

1. AXI (Advanced eXtensible Interface) :- It is used for Higher Performance Low Latency and High Bandwidth Applications.
2. AHB (Advanced High-Performance Bus) :- It is used for High Speed and High Bandwidth Application.
3. APB (Advanced Peripheral Bus) :- It is used for low power , lower Bandwidth Applications.

Overview of AHB Lite Protocol:

The Advanced High-performance Bus Lite (AHB-Lite) is a subset of the ARM AMBA (Advanced Microcontroller Bus Architecture) specification. It is designed for high-performance, high-clock-frequency system modules. Unlike the full AHB specification, AHB-Lite is optimized for a single-master design, which eliminates the need for complex arbitration logic, request/grant signals, and split transactions. This makes it highly efficient for connecting a single CPU or DMA controller to on-chip memories and high-speed peripherals, most common AHB Lite slaves are Internal Memory Devices, External Memory Interfaces and High Bandwidth Peripherals.

Features Required for High Performance:

1. **Burst Transfer:-** Using Burst Transfer we can send more data. It is a High-Performance Bus Operation that sends Multiple Data Packets using a single Initial Address Cycle through Incrementing or Wrapping Methods. It uses Pipelining to Overlap Address and Data Phases Improving overall Data Throughput.
2. **Single clock edge operation:-** Everything will Happen at the Posedge of Clock Signal.
3. **Non Tristate Implementation:-** AHB lite Uses Dedicated point to point connections with Multiplexers rather than shared tristate wires. This means signals are driven high or low continuously by a single source avoiding high Impedance states on the Chip.
4. **Wide Data Bus Configuration:-** 32, 64, 128, 256, 512, 1024 Bits

Operation of AHB protocol

The core operation of AHB-Lite is based on a strictly synchronous, two-phase pipeline mechanism. Every transaction is divided into an Address Phase and a Data Phase.

1. **Address Phase** (Phase 1): The master drives the address (HADDR) and control signals (such as transfer type HTRANS, direction HWRITE, and size HSIZE) onto the bus. This phase strictly lasts for a single clock cycle.
2. **Data Phase** (Phase 2): The actual data transfer occurs. For a write transaction, the master drives the write data (HWDATA); for a read transaction, the addressed slave drives the read data (HRDATA). This phase can last for one clock cycle or be extended by the slave using wait states.

Pipelining

To maximize bus throughput, AHB-Lite overlaps the Address and Data phases of consecutive transfers. While the Data Phase of the current transfer is being executed, the Address Phase of the *next* transfer is simultaneously broadcast on the bus. This pipelining

ensures that a new data payload can be transferred on every single clock cycle during burst operations.

Wait State Insertion (HREADY)

Because different peripherals operate at different speeds, slaves must have a mechanism to stall the bus master if they cannot supply or accept data immediately.

- The slave uses the HREADYOUT signal to insert wait states.
- If HREADYOUT is driven **LOW (0)**, the current Data Phase is extended, and the master is forced to hold the data (for writes) or wait for the data (for reads).
- Because of pipelining, stalling the current Data Phase automatically stalls the Address Phase of the subsequent transfer.
- When the slave is ready to complete the transaction, it drives HREADYOUT **HIGH (1)**, allowing the bus to sample the data and move to the next pipeline stage.

Transfer Types (HTRANS)

The master uses the 2-bit HTRANS signal during the Address Phase to indicate the nature of the current transfer:

- **IDLE (00):** Indicates that no data transfer is required. Slaves must provide a zero-wait-state OKAY response.
- **BUSY (01):** Allows the master to insert idle cycles in the middle of a burst transfer. The slave ignores the data phase.
- **NONSEQ (10):** Indicates a single transfer or the first transfer of a burst. The address is unrelated to the previous transfer.
- **SEQ (11):** Indicates the remaining transfers in a burst. The address is sequentially related to the previous transfer.

Slave Responses (HRESP)

The slave must provide a status response during the Data Phase using the 1-bit HRESP signal

- **OKAY (0):** Indicates the transfer completed successfully. It is also the mandatory response for IDLE or BUSY transfers.
- **ERROR (1):** Indicates a failure occurred during the transfer (e.g., attempting to write to read-only memory or accessing an unmapped address region).
- **The Two-Cycle Error Protocol:** To allow the pipelined master enough time to cancel the subsequent address phase, an ERROR response requires a strict two-cycle handshake. In the first cycle, the slave drives HRESP to ERROR and HREADYOUT to LOW. In the second cycle, it holds HRESP at ERROR but drives HREADYOUT to HIGH to finally retire the transfer.

Overview of APB4 Protocol:

The Advanced Peripheral Bus (APB4) is a highly simplified, low-power AMBA protocol designed to interface with low-bandwidth peripherals (such as timers, UARTs, and GPIOs). In this system architecture, the AHB2APB bridge acts as the APB Master, translating high-speed, pipelined AHB-Lite transactions into standard APB4 transfers. The bridge is highly configurable and routes transactions to a parameterizable number of APB slaves using a dedicated address decoder.

Unlike the pipelined AHB-Lite bus, APB4 relies on a strictly non-pipelined, two-phase state machine to perform data transfers. The bridge's APB Master FSM controls this flow by explicitly moving through IDLE, SETUP, and ACCESS states, ensuring that the enable signal (PENABLE) is asserted only in the ACCESS cycle.

- **IDLE State:** This is the default resting phase of the bus. PSEL and PENABLE are both held LOW, indicating no active transfer.
- **Phase 1: SETUP Cycle:** When a valid transfer request crosses the asynchronous clock boundary from the AHB domain, the APB FSM enters the SETUP state.
    - The bridge drives the target address (PADDR), transfer direction (PWRITE), and write data (PWDATA).
    - It asserts the one-hot slave select signal (PSEL[n]) for the specific targeted peripheral based on the address decode map.
    - PENABLE strictly remains LOW during this cycle.
- **Phase 2: ACCESS Cycle:** On the very next clock edge, the FSM transitions to the ACCESS state.
    - PENABLE is asserted HIGH to inform the slave that the address and data on the bus are stable and should be captured or acted upon.

- All other signals (PSEL, PADDR, PWRITE, PWDATA) are held perfectly constant.

3. Wait-State Insertion and System Protection

During the ACCESS cycle, the targeted APB slave governs when the transaction completes using the PREADY signal.

- **Stretching (Wait-States):** If the slave requires more time to process the request, it holds PREADY LOW. The bridge remains frozen in the ACCESS state, which in turn stretches the AHB transaction by holding HREADYOUT LOW.
- **The Timeout Monitor:** Because APB stall cycles directly halt the high-speed CPU on the AHB bus, a malfunctioning APB slave holding PREADY LOW indefinitely would cause a complete system hang. To prevent this, the bridge implements a programmable timeout monitor. It continuously counts the cycles during slave-driven PREADY stretching. If the wait time exceeds the parameterized threshold, the bridge forcefully aborts the APB transfer, releases the FSM back to IDLE, and returns an error to the AHB master.

4. Byte-Lane Handling (PSTRB)

To support partial-word writes, the APB4 standard introduces the PSTRB (Write Strobes) signal.

- The bridge dynamically generates the 4-bit PSTRB mask by translating the AHB transfer size (HSIZE) and the lower two address bits (HADDR[1:0]).
- This ensures that if the AHB master performs a single-byte or half-word write, the APB slave only updates the specifically targeted byte lanes in its 32-bit memory, preserving the existing data in the untouched lanes. During read operations, PSTRB is driven to zero.

5. Error Handling (PSLVERR)

If an APB slave encounters a local fault during a transaction (e.g., an access violation or an invalid register write), it asserts the PSLVERR signal during the ACCESS cycle while PREADY is HIGH.

- The bridge's response mapper captures this PSLVERR signal.
- It then synchronizes this status back across the clock domain and accurately maps it into the strict two-cycle AHB ERROR response (HRESP), ensuring the CPU is properly notified of the peripheral-level fault without violating the AHB-Lite protocol timing.

1. ## Microarchitecture Specification

   The design is a fully asynchronous AHB-Lite to APB4 bridge, enabling communication between a high-performance AHB manager and parameterized lower-speed APB peripherals.

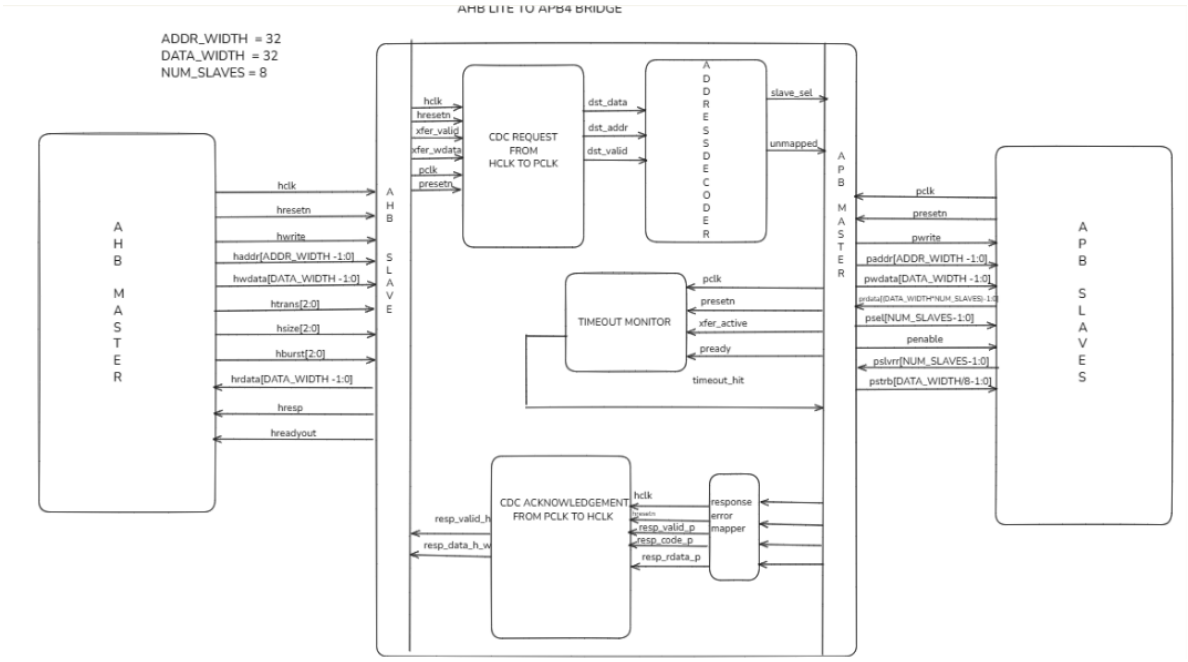
- **AHB Capture (FSM):** Operates in the hclk domain. It absorbs AHB transfers (address, control, and write data), drives hreadyout low to stall the AHB bus, and initiates a cross-domain request.

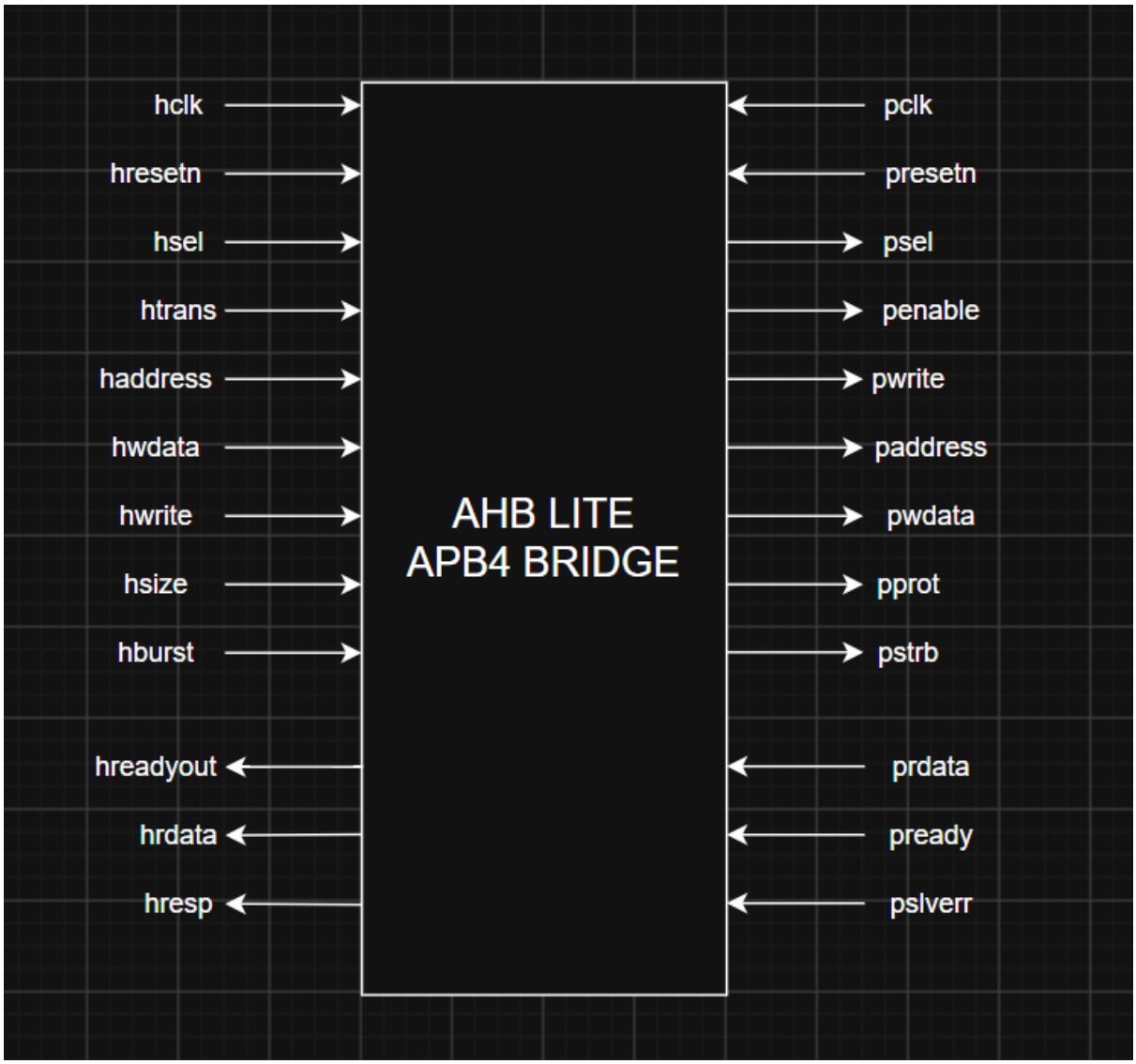
- **CDC Request (hclk to pclk):** A multi-bit payload synchronizer using a toggle-based, 2-phase handshake protocol to safely transfer the command, address, and data across the asynchronous boundary.
- **Address Decoder:** Evaluates the synchronized address to generate a one-hot slave_sel signal. It asserts an unmapped flag if the address targets a region outside the configured slave memory map.

- **APB Master (FSM):** Operates in the pclk domain. It translates the synchronized request into a standard APB4 SETUP and ACCESS sequence, generating byte strobes (pstrb) and managing slave selection.

- **Timeout Monitor:** Prevents system deadlocks by counting cycles while xfer_active is asserted. If an APB slave fails to assert pready within the parameterized TIMEOUT_CYCLES, it forces an error response.

- **Response Error Mapper & CDC Ack:** Operates in the pclk domain to pack the APB read data and pslverr status into a response payload. This is passed back to the hclk domain via a secondary CDC handshake block to complete the AHB transfer.


AHB LITE TO APB4 BRIDGE
ADDR_WIDTH = 32
DATA_WIDTH = 32
NUM_SLAVES = 8
AHB MASTER
hclk
hresetn
hwrite
haddr[ADDR_WIDTH -1:0]
hwdata[DATA_WIDTH -1:0]
htrans[2:0]
hsize[2:0]
hburst[2:0]
hrdata[DATA_WIDTH -1:0]
hresp
hreadyout
AHB SLAVE
CDC REQUEST FROM HCLK TO PCLK
hclk
hresetn
xfer_valid
xfer_wdata
pclk
presetn
dst_data
dst_addr
dst_valid
ADDRESS DECODER
slave_sel
unmapped
APB MASTER
TIMEOUT MONITOR
pclk
presetn
xfer_active
pready
timeout_hit
CDC ACKNOWLEDGEMENT FROM PCLK TO HCLK
hclk
resp_valid_p
resp_code_p
resp_rdata_p
response error mapper
pclk
presetn
pwrite
paddr[ADDR_WIDTH -1:0]
pwdata[DATA_WIDTH -1:0]
psel[NUM_SLAVES-1:0]
penable
pslvrr[NUM_SLAVES-1:0]
pstrb[DATA_WIDTH/8-1:0]
APB SLAVES

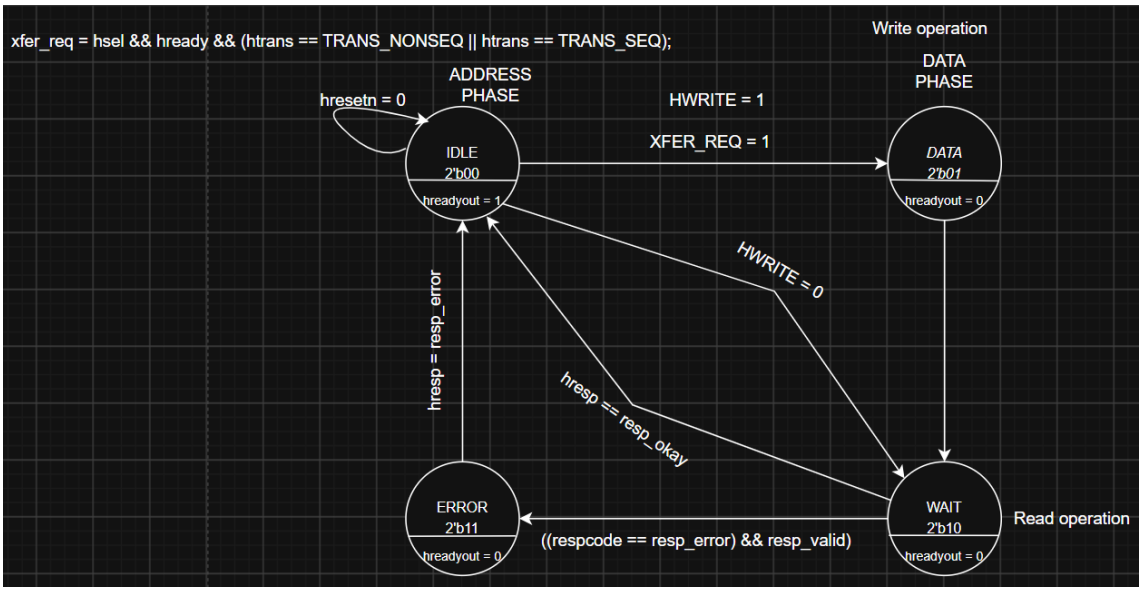


Finite State Machines (FSM)

AHB Capture FSM State Diagram:

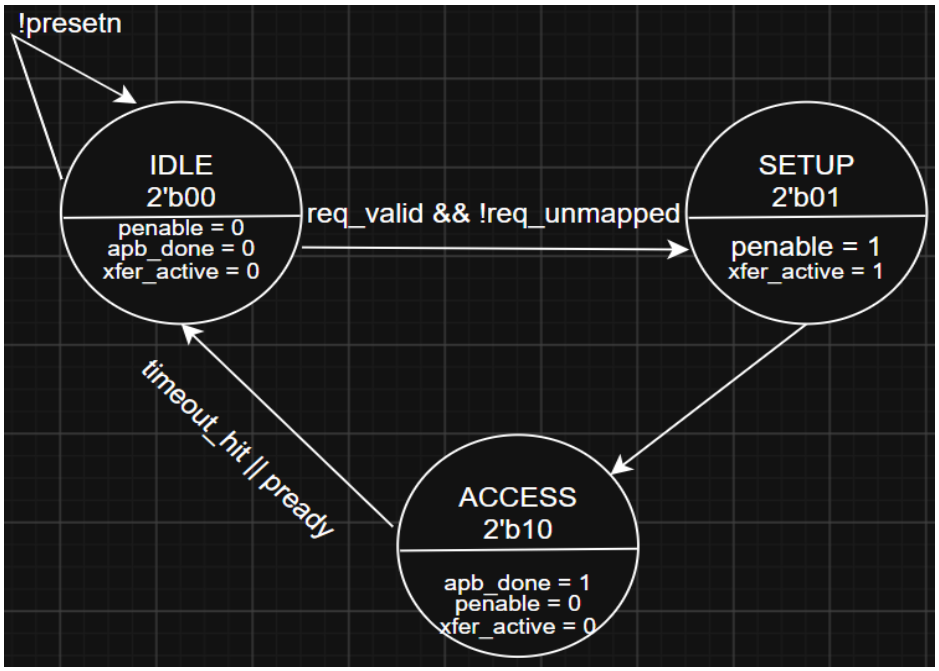
- **IDLE (2'b00):** The default state (hreadyout = 1). Transitions to DATA when a valid non-sequential or sequential transfer is requested (xfer_req = 1).
- **DATA (2'b01):** The AHB data phase. The bus is stalled (hreadyout = 0) while data is captured. Unconditionally transitions to WAIT.
- **WAIT (2'b10):** The bridge waits for the CDC acknowledgment from the APB domain. Transitions back to IDLE upon successful completion (resp_valid == 1 and respcode == resp_okay). If a slave error or timeout occurs, it transitions to ERROR.

- **ERROR (2'b11):** Asserts the two-cycle AHB error response (hresp = resp_error). Unconditionally transitions back to IDLE.



APB Master FSM State Diagram:

- **IDLE (2'b00):** Default state where penable = 0 and xfer_active = 0. Transitions to SETUP upon receiving a valid, mapped request from the CDC synchronizer (req_valid && !req_unmapped).
- **SETUP (2'b01):** The APB setup phase. psel is asserted, and penable is driven high. Unconditionally transitions to ACCESS on the next clock edge (xfer_active = 1).
- **ACCESS (2'b10):** The APB access phase. The FSM waits for the peripheral. It transitions back to IDLE when the slave asserts pready, or if the timeout_hit signal is asserted. Upon exit, apb_done is flagged to trigger the CDC acknowledgment.


!presetn
IDLE
2'b00
penable = 0
apb_done = 0
xfer_active = 0
req_valid && !req_unmapped
SETUP
2'b01
penable = 1
xfer_active = 1
timeout_hit || pready
ACCESS
2'b10
apb_done = 1
penable = 0
xfer_active = 0

Datapath Widths

The bridge is designed with parameterized datapath routing, allowing for scalability, though it defaults to standard 32-bit AMBA specifications.

- **Address Bus Width (ADDR_WIDTH):** Configured for 32 bits. The full 32-bit address (haddr) is captured from the AHB master and safely passed across the CDC boundary to drive the APB peripheral address (paddr).
- **Data Bus Width (DATA_WIDTH):** Configured for 32 bits. This applies to both the write datapath (hwdata to pwdata) and the read datapath (prdata multiplexed back to hrdata).
- **Byte Lane Strobing:** The APB4 protocol requires a write strobe (pstrb). The datapath includes a translation function that decodes the 3-bit AHB transfer size (hsize) into a 4-bit byte strobe array (DATA_WIDTH/8), ensuring narrow transfers (e.g., 8-bit or 16-bit writes) only update the valid byte lanes on the 32-bit APB slave.
- **Peripheral Multiplexing:** The read datapath aggregates a flattened bus from all slaves (prdata_bus of size DATA_WIDTH * NUM_SLAVES). The address decoder outputs a selection signal to multiplex the correct 32-bit slice back to the master.

2. Clock and Reset Architecture

The bridge acts as an asynchronous boundary between a high-performance processor/matrix and slower peripherals, requiring strict clock and reset domain isolation.

- **Clock Domains:** The design operates on two completely independent, asynchronous clocks: hclk (AHB-Lite Domain) and pclk (APB4 Domain). There is no assumed phase or frequency relationship between them.
- **Domain Isolation:** All cross-domain control and data signals are strictly routed through the 2-phase toggle handshake synchronizers (cdc_handshake). The AHB FSM operates strictly on hclk, and the APB FSM operates strictly on pclk.
- **Reset Signals:** The architecture utilizes two independent reset networks: hresetn (for the AHB domain) and presetn (for the APB domain).
- **Reset Polarity and Type:** Both resets are active-low and operate asynchronously, immediately driving the FSMs to their IDLE states and clearing all pending request payloads or response codes upon assertion.

- **Reset Synchronization:** As flagged in your SpyGlass CDC report, these resets are handled independently within their domains to ensure they do not cause metastability when deasserted relative to their respective clocks.

2. Register Map

2.1 Slave 0 Base Memory Map

The apb_led_slave module is mapped to the base address space of Slave 0 (psel[0]). It acts as a memory-mapped output controller to drive the physical LEDs on the FPGA development board.

Offset	Register Name	Width	Access	Reset Value	Description
0x00	LED_CTRL	32-bit	RW	0x00000000	LED Control and Status Register.

2.2 LED_CTRL Bit-Field Description

Bits	Field Name	Access	Reset	Description
[31:8]	RSVD	RW	0x0	**Reserved / Scratchpad.** These bits are implemented in the internal register and support byte-strobe writes, but do not drive any external hardware.
[7:0]	LED_OUT	RW	0x0	**LED Output Data.** Data written to this byte lane directly drives the 8 physical external LEDs on the FPGA board. A logic 1 turns the corresponding LED on.

2.3 Slave Interface Characteristics

- **Wait States:** Zero wait states. The slave always asserts pready_bus = 1'b1.
- **Error Responses:** Error signaling is disabled. The slave permanently drives pslverr_bus = 1'b0.

- **Byte Lane Support:** Full byte-strobe (pstrb) support is implemented, allowing the AHB master to perform 8-bit, 16-bit, or 32-bit writes to the LED register safely.

3. RTL Source

```
AHB_APB_BRIDGE/
├── sim/
│   └── tb_ahb2apb_bridge.v
├── src/
│   ├── addr_decoder.v
│   ├── ahb2apb_bridge.v
│   ├── ahb_capture_fsm.v
│   ├── apb_master_fsm.v
│   ├── cdc_handshake.v
│   ├── resp_error_mapper.v
│   ├── timeout_monitor.v
│   └── bridge.sgdc
└── syn/
    ├── log/
    │   └── dc_shell.log
    ├── mapped/
    │   └── (Mapped design in .ddc format)
    ├── netlist/
    │   └── (Final gate-level netlist mapped to SAED 32nm library)
    ├── report/
    │   └── (Area, timing, and QoR reports)
    ├── script/
    │   ├── setup.tcl & run.tcl
    │   └── constraints_sdc.sdc
    └── unmapped/
        └── (GTECH generic library design in .ddc format)
```

sim/ (Simulation): Contains tb_ahb2apb_bridge.v, which is the self-checking testbench driving the directed AHB stimulus and validating the APB endpoint responses.

src/ (Source Code): Houses the complete SystemVerilog/Verilog RTL file list for the bridge. It also contains bridge.sgdc, the SpyGlass Design Constraints file defining the independent clocks (hclk, pclk) and resets used for structural CDC verification.

syn/ (Synthesis): The backend workspace for Synopsys Design Compiler (dc_shell).

- script/: Contains setup.tcl (loads the SAED 32nm HVT/LVT target technology libraries and sets up search paths) and run.tcl (the primary elaboration, compilation, and optimization script) along with synopsys design constraints SDC for the synthesis and for timing report.
- unmapped/ & mapped/: Stores the intermediate GTECH .ddc databases and the technology-mapped .ddc databases, respectively.
- netlist/: Contains the final compiled gate-level structural netlist.
- report/ & log/: Captures all analysis reports (area, timing) and terminal execution logs for debugging.

Run.tcl

```
set design "ahb2apb_bridge"

analyze -format verilog {apb_master_fsm.v ahb2apb_bridge.v ahb_capture_fsm.v timeout_monitor.v resp_error_mapper.v cdc_handshake.v addr_decoder.v }

elaborate $design

source constraints_sdc.sdc

link

check_design

write -format verilog -hier -output unmapped/$design.v

write -format ddc -hierarchy -output unmapped/$design.ddc

compile_ultra

check_timing

write -format verilog -hierarchy -output netlist/netlist_$design.v

write -format ddc -hierarchy -output mapped/$design.ddc

cd report
```

report_timing > timing_report_$design.rpt

report_area > area_report_$design.rpt

cd ..

Setup.tcl

set search_path [list . ../src ./script /fetools/work_area/frontend/SAED32_EDK/lib/stdcell_lvt/db_nldm]

set target_library [list saed32lvt_ss0p75v25c.db]

set link_library "* $target_library "

Constraints_sdc.sdc

create_clock -name hclk -period 10 [get_ports hclk]

create_clock -name pclk -period 40 [get_ports pclk]

set_clock_groups -asynchronous -group [get_clocks hclk] -group [get_clocks pclk]

set_clock_uncertainty -setup 0.2 [get_clocks pclk]

set_clock_uncertainty -setup 0.2 [get_clocks hclk]

set_input_delay -clock hclk -max 3 [get_ports {hsel hready haddr htrans hwrite hsize hwdata}]

set_input_delay -clock hclk -min 0.5 [get_ports {hsel hready haddr htrans hwrite hsize hwdata}]

set_output_delay -clock hclk -min 2 [get_ports {hrdata hreadyout hresp}]

set_output_delay -clock hclk -max 3 [get_ports {hrdata hreadyout hresp}]

set_input_delay -clock pclk -max 12 [get_ports {prdata_bus pready_bus pslverr_bus}]

set_input_delay -clock pclk -min 2 [get_ports {prdata_bus pready_bus pslverr_bus}]

set_output_delay -clock pclk -min 2 [get_ports {paddr pwrite pwdata pstrb}]

set_output_delay -clock pclk -max 12 [get_ports {paddr pwrite pwdata pstrb}]

4. Self-checking testbench

A self-checking Verilog testbench (tb_ahb2apb_bridge.v) was developed to perform baseline directed testing. As comprehensive constrained-random verification is outside the scope of this deliverable, the testbench focuses strictly on proving basic protocol legality, datapath integrity, and edge-case recovery (e.g., errors and timeouts).

The testbench environment includes:

- Asynchronous Clock Generation: Independent clock generators for hclk (period: 7.4ns) and pclk (period: 10.0ns) to emulate real-world asynchronous phase relationships.
- Emulated APB Slave: A behavioral 2KB memory array (slave_mem) that supports configurable wait states, error injection (pslverr), and timeout emulation.
- Automated Checking: Encapsulated verification tasks (e.g., check_read, check_write) that automatically compare the bridged read data and hresp statuses against expected values, incrementing a global pass/fail counter.

Directed Test Scenarios

Test ID	Scenario Description	Verification Objective	Result
Test 1	Zero-wait-state writes and reads	Verify basic datapath integrity and fundamental read/write legality.	PASS
Test 2	Interleaved writes and reads	Ensure the FSM transitions correctly between changing transfer directions.	PASS
Test 3	Back-to-back transfers with APB wait states	Verify the bridge safely stalls the AHB bus when the APB slave asserts pready=0.	PASS
Test 4	Read of uninitialized memory	Ensure unwritten locations return the expected zeroized reset value.	PASS

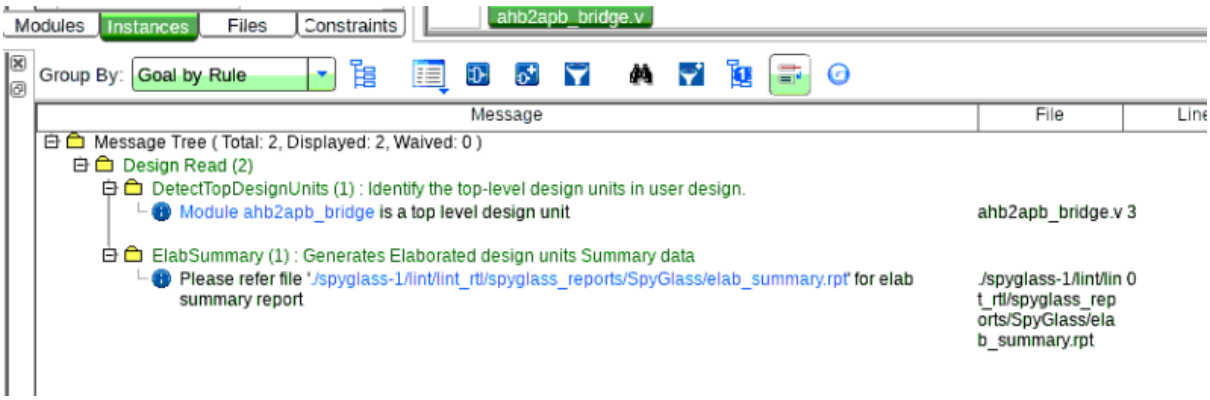
Test ID	Scenario Description	Verification Objective	Result
Test 5	Slave timeout emulation	Trigger the internal watchdog by forcing a slave to never respond. Verify HRESP=ERROR.	PASS
Test 6	Post-timeout bus recovery	Prove the bridge FSM safely returns to IDLE and accepts new transfers after a timeout.	PASS
Test 7	Pipelined back-to-back writes	Verify zero-bubble pipelined AHB transfers are captured and executed without data loss.	PASS
Test 8	Sub-word byte writes (PSTRB)	Validate byte-lane strobing ensuring adjacent bytes in the 32-bit word are not corrupted.	PASS
Test 9	APB slave error (PSLVERR) mapping	Verify a peripheral error is correctly translated into the strict 2-cycle AHB ERROR response.	PASS
Test 10	Unmapped address decode error	Access an out-of-bounds address (0x0080_0000) to verify fast internal error generation.	PASS
Test 11	Post-decode-error bus recovery	Prove the FSM clears the internal error state and completes subsequent valid transfers.	PASS
Test 12	AHB-side HREADY wait states	Emulate master-side bus stalls to ensure the bridge holds data and address correctly.	PASS

Test ID	Scenario Description	Verification Objective	Result
Test 13	Combined AHB & APB wait states	Stress test the CDC synchronizers and FSMs under maximum backpressure from both sides.	PASS

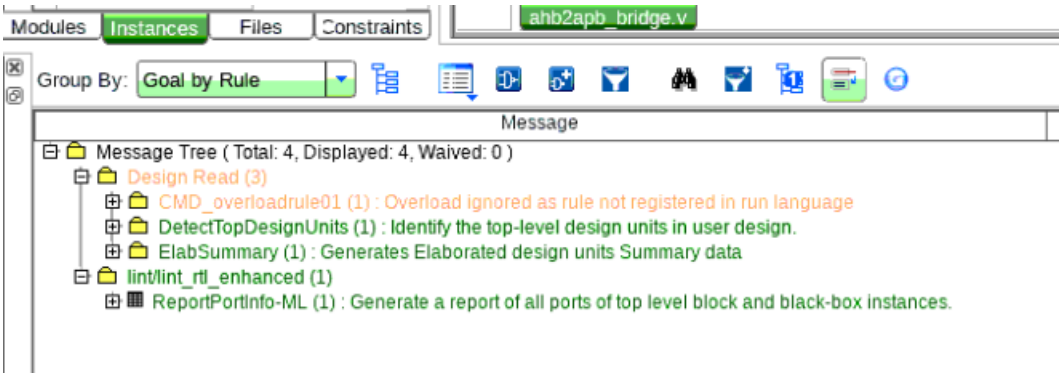
5. Lint Report + waiver File

5.1 **Lint Report Summary** The RTL design was analyzed using Synopsys SpyGlass. The basic structural read and elaboration (Design Read goal) completed cleanly with no errors, successfully identifying ahb2apb_bridge as the top-level design unit.

Lint_rtl



Lint_rtl_enhanced

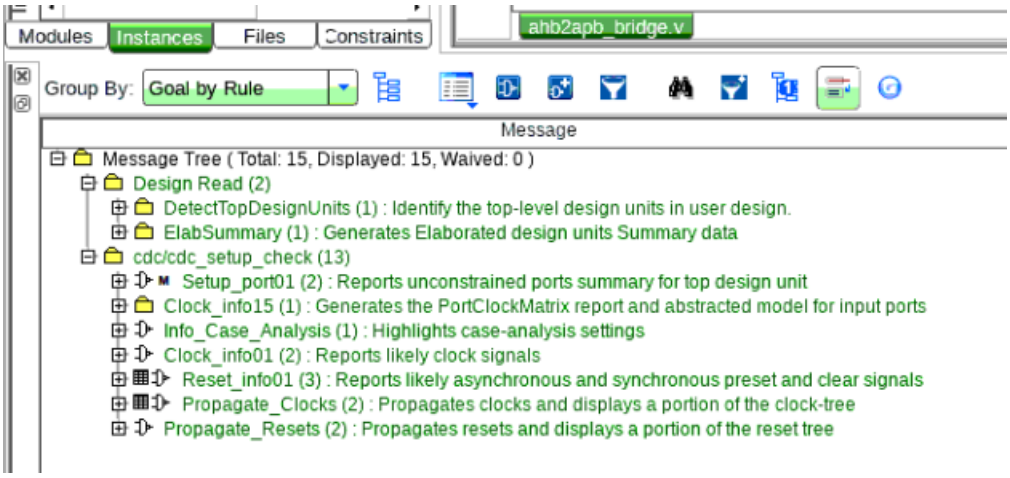


6. CDC report + CDC inventory

The design was verified for structural and functional clock domain crossing safety using the Synopsys SpyGlass CDC flow. The analysis was conducted in two primary phases, both of which completed cleanly with no structural violations or data loss conditions detected.

CDC Setup Check (cdc_setup_check): The setup phase successfully identified and propagated the asynchronous clock domains (hclk and pclk) and their respective asynchronous resets (hresetn and presetn). Rules such as Clock_info01, Reset_info01, and Propagate_Clocks/Resets passed, confirming the .sgdc constraints correctly map to the RTL.

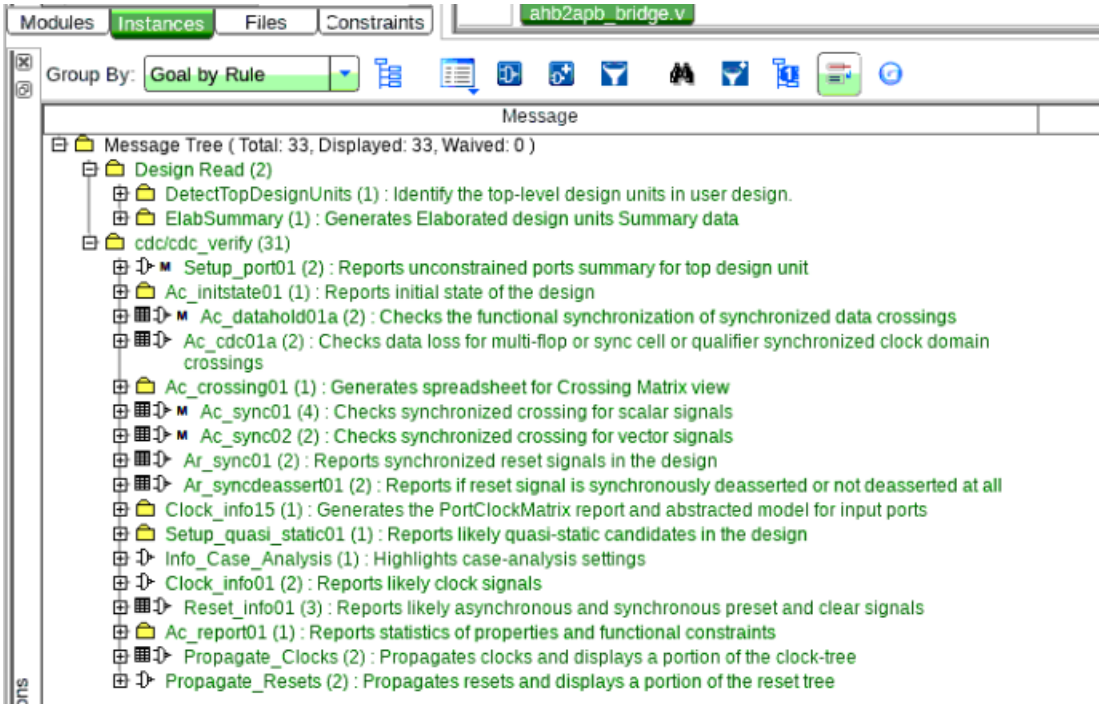
cdc_setup



CDC Verification (cdc_verify): The functional verification phase rigorously checked the synchronizers. Crucially, rule Ac_datahold01a passed, confirming that the multi-bit payloads are held stable for a sufficient number of clock cycles while the control signals cross the asynchronous boundary. Rules Ac_sync01 and Ac_cdc01a verified that scalar

control signals are safely synchronized without risk of data loss or metastability propagation.

cdc_verify



CDC Inventory

The bridge architecture isolates the fast AHB domain (hclk) from the slow APB domain (pclk) using strict boundary synchronizers. All asynchronous crossings are cataloged below.

Crossing Name	**Source Domain**	**Dest. Domain**	**Classification**	**Synchronizing Structure**
Request Payload (u_cdc_req)	hclk	pclk	Multi-bit Data & Control (Address, Write Data,	**2-Phase Toggle Handshake.** Control signals utilize a toggle

Crossing Name	Source Domain	Dest. Domain	Classification	Synchronizing Structure
			Size, Write Enable)	flip-flop synchronized via a standard 2-stage standard synchronizer (double-flop). The multi-bit datapath uses a stable payload holding register, governed by the synchronized toggle signal to prevent multi-bit skew.
Response Payload (u_cdc_ack)	pclk	hclk	Multi-bit Data & Status (Read Data, Response Error Code)	**2-Phase Toggle Handshake.** Identical structure to the Request CDC. The APB completion status toggles a control bit, synchronized via a 2-stage double-flop into the hclk domain, which then safely samples the static read data/error payload.
AHB Reset (hresetn)	Asynchronous Pad	hclk	Asynchronous Reset	Driven asynchronously,

Crossing Name	Source Domain	Dest. Domain	Classification	Synchronizing Structure
				evaluated by SpyGlass rules Ar_sync01 and Ar_syncdeassert01 for safe localized domain application.
APB Reset (presetn)	Asynchronous Pad	pclk	Asynchronous Reset	Driven asynchronously, evaluated by SpyGlass rules Ar_sync01 and Ar_syncdeassert01 for safe localized domain application.

7. **SDC constraints file**

Constraints_sdc.sdc

```
create_clock -name hclk -period 10 [get_ports hclk]
create_clock -name pclk -period 40 [get_ports pclk]
set_clock_groups -asynchronous -group [get_clocks hclk] -group [get_clocks pclk]
set_clock_uncertainty -setup 0.2 [get_clocks pclk]
set_clock_uncertainty -setup 0.2 [get_clocks hclk]
set_input_delay -clock hclk -max 3 [get_ports {hsel hready haddr htrans hwrite hsize hwdata}]
set_input_delay -clock hclk -min 0.5 [get_ports {hsel hready haddr htrans hwrite hsize hwdata}]
set_output_delay -clock hclk -min 2 [get_ports {hrdata hreadyout hresp}]
set_output_delay -clock hclk -max 3 [get_ports {hrdata hreadyout hresp}]
```

set_input_delay -clock pclk -max 12 [get_ports {prdata_bus pready_bus pslverr_bus}]
set_input_delay -clock pclk -min 2 [get_ports {prdata_bus pready_bus pslverr_bus}]
set_output_delay -clock pclk -min 2 [get_ports {paddr pwrite pwdata pstrb}]
set_output_delay -clock pclk -max 12 [get_ports {paddr pwrite pwdata pstrb}]

SDC Constraints

Constraint Type	**SDC Command / Target**	**Description**
Clocks	create_clock -name hclk -period 10 [get_ports hclk]	Defines the high-speed AHB matrix clock at 100 MHz (10ns period) to drive AHB_Capture_FSM logic.
Clocks	create_clock -name pclk -period 40 [get_ports pclk]	Defines the slower peripheral APB clock at 25 MHz (40ns period) to drive the slave-side FSM and logic.
Async Clock Groups	set_clock_groups -asynchronous -group [get_clocks hclk] -group [get_clocks pclk]	Instructs the STA engine that hclk and pclk are completely asynchronous. This inherently masks false paths across the CDC boundaries, preventing the tool from over-optimizing the synchronizers.
Clock Uncertainty	set_clock_uncertainty -setup 0.2 [get_clocks hclk]	Models a 0.2ns margin for clock jitter and post-CTS clock skew. This enforces a stricter setup timing check during pre-layout synthesis.

Constraint Type	SDC Command / Target	Description
	set_clock_uncertainty -setup 0.2 [get_clocks pclk]	
Input Delay (Max)	set_input_delay -clock hclk -max 3 [get_ports {hsel hready haddr htrans hwrite hsize hwdata}]	Models the maximum external setup delay (3ns) of the upstream AHB Master, leaving 7ns for the bridge's input logic.
Input Delay (Min)	set_input_delay -clock hclk -min 0.5 [get_ports {hsel hready haddr htrans hwrite hsize hwdata}]	Models the minimum external hold delay (0.5ns) to constrain hold timing analysis on the bridge's input flops.
Output Delay (Max)	set_output_delay -clock hclk -max 3 [get_ports {hrdata hreadyout hresp}]	Reserves 3ns of the 10ns clock cycle for the downstream AHB Master's logic to meet its setup requirements.
Output Delay (Min)	set_output_delay -clock hclk -min 2 [get_ports {hrdata hreadyout hresp}]	Models the minimum external hold requirement (2ns) for downstream AHB logic.
Input Delay (Max)	set_input_delay -clock pclk -max 12 [get_ports {prdata_bus pready_bus pslverr_bus}]	Allocates up to 12ns for the APB slaves to decode and drive responses, leaving 28ns for the bridge's internal capture logic.
Input Delay (Min)	set_input_delay -clock pclk -min 2 [get_ports {prdata_bus pready_bus pslverr_bus}]	Defines the minimum external delay (2ns) for APB slave responses to check hold timing on the bridge's input flops.

Constraint Type	SDC Command / Target	Description
Output Delay (Max)	set_output_delay -clock pclk -max 12 [get_ports {paddr pwrite pwdata pstrb}]	Ensures the bridge outputs data early enough to account for up to 12ns of external routing and APB slave setup requirements.
Output Delay (Min)	set_output_delay -clock pclk -min 2 [get_ports {paddr pwrite pwdata pstrb}]	Sets the minimum external hold time requirement (2ns) for the downstream APB slave inputs.

8. **Synthesis Report**
   check_design

```
****************************************
check_design summary:
Version:     S-2021.06-SP4
Date:        Sun Sep  6 18:43:07 2026
****************************************

                 Name                                                  Total
--------------------------------------------------------------------------------
Inputs/Outputs                                                         37
    Unconnected ports (LINT-28)                                         1
    Shorted outputs (LINT-31)                                          17
    Constant outputs (LINT-52)                                         19
--------------------------------------------------------------------------------

Warning: In design 'ahb2apb_bridge', port 'htrans[0]' is not connected to any nets. (LINT-28)
Warning: In design 'ahb2apb_bridge', output port 'paddr[31]' is connected directly to output port 'paddr[30]'. (LINT-31)
Warning: In design 'ahb2apb_bridge', output port 'paddr[31]' is connected directly to output port 'paddr[29]'. (LINT-31)
Warning: In design 'ahb2apb_bridge', output port 'paddr[31]' is connected directly to output port 'paddr[28]'. (LINT-31)
Warning: In design 'ahb2apb_bridge', output port 'paddr[31]' is connected directly to output port 'paddr[27]'. (LINT-31)
Warning: In design 'ahb2apb_bridge', output port 'paddr[31]' is connected directly to output port 'paddr[26]'. (LINT-31)
Warning: In design 'ahb2apb_bridge', output port 'paddr[31]' is connected directly to output port 'paddr[25]'. (LINT-31)
Warning: In design 'ahb2apb_bridge', output port 'paddr[31]' is connected directly to output port 'paddr[24]'. (LINT-31)
Warning: In design 'ahb2apb_bridge', output port 'paddr[31]' is connected directly to output port 'paddr[23]'. (LINT-31)
Warning: In design 'ahb2apb_bridge', output port 'paddr[31]' is connected directly to output port 'paddr[22]'. (LINT-31)
Warning: In design 'ahb2apb_bridge', output port 'paddr[31]' is connected directly to output port 'paddr[21]'. (LINT-31)
Warning: In design 'ahb2apb_bridge', output port 'paddr[31]' is connected directly to output port 'paddr[19]'. (LINT-31)
Warning: In design 'ahb2apb_bridge', output port 'paddr[31]' is connected directly to output port 'paddr[18]'. (LINT-31)
Warning: In design 'ahb2apb_bridge', output port 'paddr[31]' is connected directly to output port 'paddr[17]'. (LINT-31)
Warning: In design 'ahb2apb_bridge', output port 'paddr[31]' is connected directly to output port 'paddr[16]'. (LINT-31)
Warning: In design 'ahb2apb_bridge', output port 'paddr[31]' is connected directly to output port 'paddr[15]'. (LINT-31)
Warning: In design 'ahb2apb_bridge', output port 'paddr[31]' is connected directly to output port 'paddr[14]'. (LINT-31)
Warning: In design 'ahb2apb_bridge', output port 'paddr[31]' is connected directly to output port 'paddr[13]'. (LINT-31)
Warning: In design 'ahb2apb_bridge', output port 'paddr[31]' is connected directly to 'logic 0'. (LINT-52)
Warning: In design 'ahb2apb_bridge', output port 'paddr[30]' is connected directly to 'logic 0'. (LINT-52)
Warning: In design 'ahb2apb_bridge', output port 'paddr[29]' is connected directly to 'logic 0'. (LINT-52)
Warning: In design 'ahb2apb_bridge', output port 'paddr[28]' is connected directly to 'logic 0'. (LINT-52)
Warning: In design 'ahb2apb_bridge', output port 'paddr[27]' is connected directly to 'logic 0'. (LINT-52)
Warning: In design 'ahb2apb_bridge', output port 'paddr[26]' is connected directly to 'logic 0'. (LINT-52)
Warning: In design 'ahb2apb_bridge', output port 'paddr[25]' is connected directly to 'logic 0'. (LINT-52)
Warning: In design 'ahb2apb_bridge', output port 'paddr[24]' is connected directly to 'logic 0'. (LINT-52)
Warning: In design 'ahb2apb_bridge', output port 'paddr[23]' is connected directly to 'logic 0'. (LINT-52)
Warning: In design 'ahb2apb_bridge', output port 'paddr[22]' is connected directly to 'logic 0'. (LINT-52)
Warning: In design 'ahb2apb_bridge', output port 'paddr[21]' is connected directly to 'logic 0'. (LINT-52)
Warning: In design 'ahb2apb_bridge', output port 'paddr[20]' is connected directly to 'logic 0'. (LINT-52)
Warning: In design 'ahb2apb_bridge', output port 'paddr[19]' is connected directly to 'logic 0'. (LINT-52)
Warning: In design 'ahb2apb_bridge', output port 'paddr[18]' is connected directly to 'logic 0'. (LINT-52)
Warning: In design 'ahb2apb_bridge', output port 'paddr[17]' is connected directly to 'logic 0'. (LINT-52)
Warning: In design 'ahb2apb_bridge', output port 'paddr[16]' is connected directly to 'logic 0'. (LINT-52)
Warning: In design 'ahb2apb_bridge', output port 'paddr[15]' is connected directly to 'logic 0'. (LINT-52)
Warning: In design 'ahb2apb_bridge', output port 'paddr[14]' is connected directly to 'logic 0'. (LINT-52)
Warning: In design 'ahb2apb_bridge', output port 'paddr[13]' is connected directly to 'logic 0'. (LINT-52)
```

Area Report

Design Counts

Metric	Count
Number of ports	460
Number of nets	1584
Number of cells	1223
Number of combinational cells	762
Number of sequential cells	459
Number of macros/black boxes	0
Number of buf/inv	92
Number of references	33

Area Breakdown

Area Type	**Value**
Combinational area	1895.151819
Buf/Inv area	169.005760
Noncombinational area	3266.258745
Macro/Black Box area	0.000000
Net Interconnect area	1497.910010

Area Type	Value
Total cell area	5161.410563
Total area	6659.320574

Timing Summary

Path Group (Clock)	Path Type	Startpoint	Endpoint	Slack	Status
hclk	max (setup)	hready	u_ahb_cap/pending_addr_reg[1]	5.82	MET
pclk	max (setup)	pready_bus[2]	u_apb_fsm/apb_error_reg	26.91	MET

Timing Slack Summary (Min Delay)

Path Group (Clock)	Path Type	Startpoint	Endpoint	Slack	Status
hclk	min (hold)	u_cdc_ack/req_sync0_reg	u_cdc_ack/req_sync1_reg	0.22	MET

Path Group (Clock)	Path Type	Startpoint	Endpoint	Slack	Status
pclk	min (hold)	u_cdc_ack/ack_sync0_reg	u_cdc_ack/ack_sync1_reg	0.22	MET

Report_qor

Timing Summary by Path Group

Metric	hclk	pclk
Clock Period	10.00	40.00
Levels of Logic	6	8
Critical Path Length	0.90	0.81
Critical Path Slack (Setup)	+5.82 (MET)	+26.91 (MET)
Total Negative Slack (TNS)	0.00	0.00
No. of Violating Paths	0	0
Worst Hold Violation (WNS)	0.00	0.00
Total Hold Violation (THS)	0.00	0.00

Design Totals & DRC Summary

Metric	Setup (Max)	Hold (Min)
Worst Negative Slack (WNS)	0.00	0.00
Total Negative Slack (TNS)	0.00	0.00
Violating Paths	0	0
DRC Violations (Trans / Cap)	0 / 0	—

3. Cell Count & Area Breakdown

Category	Metric / Cell Type	Count	Area
Combinational	Logic Cells	762	1895.15
Sequential	Flip-Flops / Latches	459	3266.26
Buffers / Inverters	Buffers (39) + Inverters (53)	92	169.01
Interconnect	Nets (1584 nets)	—	1497.91
Macros	Black Boxes / Memories	0	0.00
Total Leaf Cells / Cell Area	—	1221	5161.41
Total Design Area	—	—	6659.32

9. STA Reports

Timing Slack Summary

Path Group (Clock)	Path Type	Startpoint	Endpoint	Slack	Status
hclk	max (setup)	hready	u_ahb_cap/pending_addr_reg[1]	5.82	MET

Timing Slack Summary (Min Delay)

Path Group (Clock)	Path Type	Startpoint	Endpoint	Slack	Status
hclk	min (hold)	u_cdc_ack/req_sync0_reg	u_cdc_ack/req_sync1_reg	0.22	MET

Timing Slack Summary (Min Delay - pclk)

Path Group (Clock)	Path Type	Startpoint	Endpoint	Slack	Status
pclk	min (hold)	u_cdc_ack/ack_sync0_reg	u_cdc_ack/ack_sync1_reg	0.22	MET

Timing Slack Summary (Max Delay - pclk)

Path Group (Clock)	Path Type	Startpoint	Endpoint	Slack	Status
pclk	max (setup)	pready_bus[2]	u_apb_fsm/apb_error_reg	26.91	MET

Violations Summary (hclk)

Path Group (Clock)	Path Type	Slack Condition	Status
hclk	max (setup)	< 0.00	No paths with slack less than 0.00

Violations Summary (Min Delay - hclk)

Path Group (Clock)	Path Type	Slack Condition	Status
hclk	min (hold)	< 0.00	No paths with slack less than 0.00

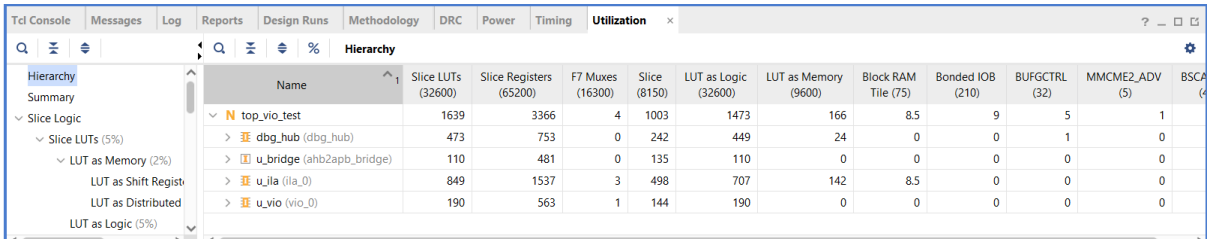
Violations Summary (Max Delay - pclk)

Path Group (Clock)	Path Type	Slack Condition	Status
pclk	max (setup)	< 0.00	No paths with slack less than 0.00

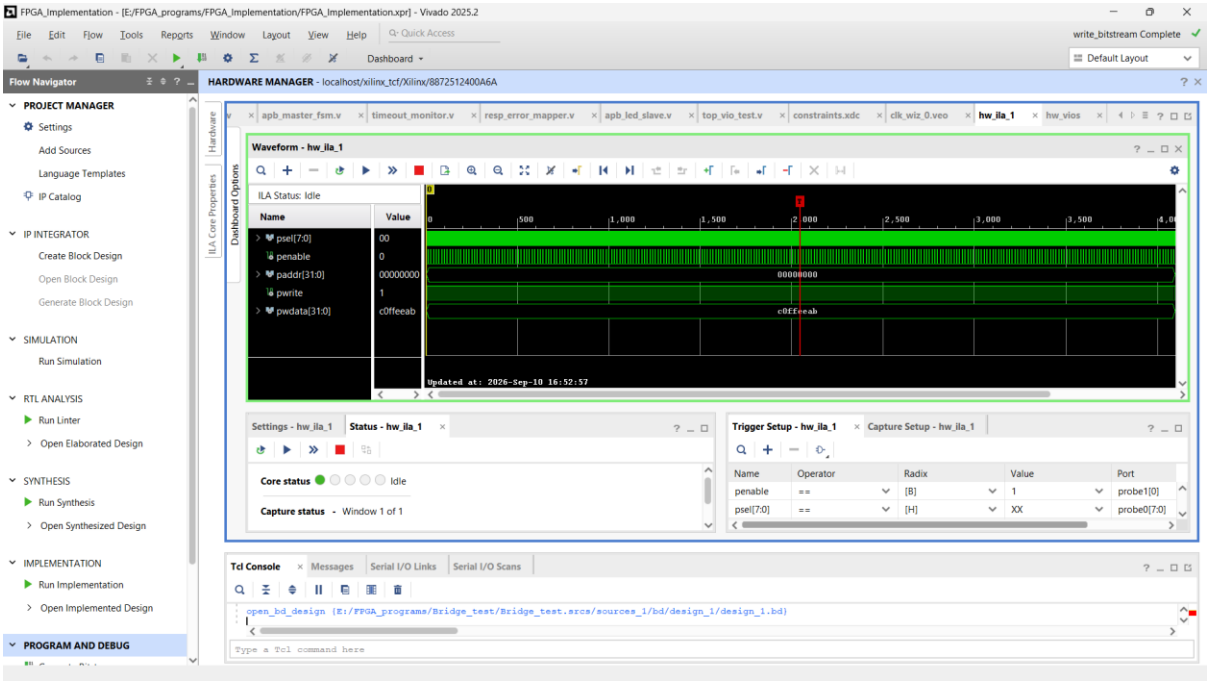
Violations Summary (Min Delay - pclk)

Path Group (Clock)	Path Type	Slack Condition	Status
pclk	min (hold)	< 0.00	No paths with slack less than 0.00

10. FPGA PROTOTYPE EVIDENCE



Write Operation



11. AI Usage Log

Date	Task	What was asked	Output accepted / modified / rejected	Time saved or lost (est.)	Errors AI introduced
Day 1	Design Requirements Analysis	Deconstruct capstone bridge requirements into architectural specifications (bus widths, parameterization, timeout thresholds, unmapped address boundaries).	Modified	Saved ~2.0 hrs	AI Error: Assumed a synchronous single-clock bridgeby default and treated unmapped addresses as non-fatal "drop/ignore" transactions rather than mandating an internal default slave and 2-cycle HRESP error response.
Day 2	Protocol Overview & Signal Rules	Clarify transfer timing rules between AHB-Lite and APB4, focusing on wait-state insertion (HREADYOUT/PREADY) and byte-strobe mechanics (PSTRB).	Modified	Saved ~1.5 hrs	AI Error: Conflated full AHB with AHB-Lite by introducing legacy multi-master signals (HBUSRET, HGRANT) and incorrectly claimed PSTRB could be omitted for 32-bit aligned accesses without risking byte-write corruption.
Day 3	Implementation Approach Selection	Evaluate architectural trade-offs between an Asynchronous FIFO vs. a 2-stage Request/Acknowledge Handshake for single-beat AHB-to-APB transfers.	Accepted	Saved ~2.0 hrs	None. Accurately reasoned that because APB4 does not support burst transfers, a registered payload with a 2-FF handshake synchronizer minimizes logic utilization without introducing throughput bottlenecks.
Day 4	Initial Draft of Core RTL Sub-blocks	Draft first-pass Verilog for modular sub-blocks: ahb_capture_fsm, addr_decoder, and apb_master_fsm.	Rejected then Modified	Saved ~2.5 hrs (Lost 30minutes debugging)	AI Error: In ahb_capture_fsm, sampled HWDATA simultaneously with HADDR in the address phase rather than waiting for the data phase. In addr_decoder, hardcoded BASE_ADDR = 0x0000_0000, omitting higher-order bitmask matching for SoC placement.
Day 5	SGDC Constraints (SpyGlass / CDC)	Draft SpyGlass Design Constraints (SGDC) for CDC verification across the hclk and pclk domains.	ModifiedSaved ~1.5 hrs	ModifiedSaved ~1.5 hrs	AI Error: Declared the entire CDC data payload bus as quasi-static rather than binding it to the handshake control qualifiers (src_valid → dst_valid), which would mask genuine data-bus skew and bus-synchronization violations

					during linting.
Day 6	SDC Constraints (STA & Synthesis)	Draft SDC timing constraints for Vivado synthesis, setting primary clocks, clock relationships, and cross-domain timing exceptions.	Modified	Saved ~1.5 hrs	AI Error: Used an overly permissive global set_false_path between hclk and pclk, which unsafely disables timing checks on critical CDC control paths. Replaced with set_clock_groups -asynchronous and explicit set_max_delay -datapath_only on payload nets.
Day 7	Testbench & Verification Stimulus	Generate directed testbench tasks, an APB slave memory model with randomized wait states, and an automated scoreboard.	Modified	Saved ~2.5 hrs	AI Error: The self-checking task sampled HRDATA immediately after de-asserting the AHB request, failing to account for asynchronous CDC latency. Modified the scoreboard to pull HREADYOUT == 1 before comparing data.
Day 8	Hardware Validation (VIO / ILA)	Generate top-level integration wrapper (top_vio_test.v) to test the bridge on physical Spartan-7 silicon without an embedded CPU.	Rejected then Modified	Saved ~1.5 hrs (Lost 30m triaging)	AI Error: Tied both hclk and pclk directly to the single board clock pin, bypassing the asynchronous CDC hardware path under test. Modified the wrapper to instantiate a Clocking Wizard (clk_wiz_0) generating 100 MHz and 33.3 MHz domains.